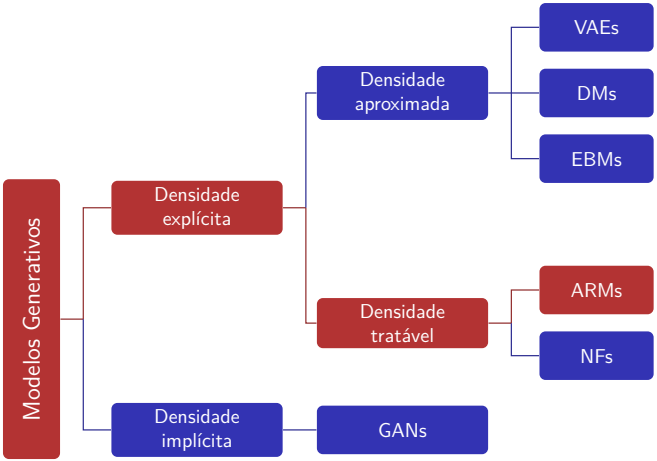
Modelos Auto-regressivos - parte 2

Inteligência Artificial Generativa

Modelos auto-regressivos [*autoregressive models*, ARMs]

Taxonomia dos modelos generativos



Modelos auto-regressivos [*autoregressive models*, ARMs]

Definição

- Modelos auto-regressivos fatoram a distribuição conjunta dos dados $\boldsymbol{x} = [x_1, x_2, \ldots, x_D]$ como um produto de distribuições condicionais:

$$p(\boldsymbol{x}) = \prod_{d=1}^{D} p(x_d \mid \underbrace{x_1, \ldots, x_{d-1}}_{\boldsymbol{x}_{<d}}) \quad \text{(regra da cadeia da probabilidade)}$$

$$= p(x_1) \prod_{d=2}^{D} p(x_d \mid \boldsymbol{x}_{<d}) \quad \text{(notação compacta)}$$

- Amostragem sequencial:
  1. sortear $x_1 \sim p(x_1)$;
  2. para $d \in \{2, \ldots, D\}$: sortear $x_d \sim p(x_d \mid \boldsymbol{x}_{<d})$.
- Aplicações: geração de texto, imagens, áudio, etc.

Como modelar as distribuições condicionais nos ARMs?

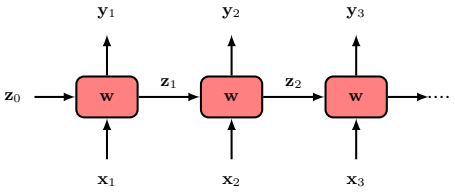
Opções de arquitetura

Podemos utilizar, por exemplo:

- Redes neurais recorrentes [*recurrent neural networks*, RNNs]
- **Transformadores** [*transformers*]
- Redes neurais convolucionais

Recapitulação

Redes neurais recorrentes



C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Exemplo de valor padrão: $z_0 = \mathbf{0}$.

Treinamento:

- Algoritmo de retropropagação através do tempo [*backpropagation through time*, BPTT]

Arquitetura recorrente clássica:

$$z_d = \phi\Big(W_{hx}x_d + W_{hz}z_{d-1} + b_z\Big)$$

$$y_d = W_{yz}z_d + b_y$$

Onde:

- z_d: estado oculto no tempo d.
- x_d: entrada no tempo d.
- W_{hx}, W_{hz}, W_{yz}: matrizes de pesos.
- b_z, b_y: vieses (*bias*).
- $\phi(\cdot)$: não-linearidade.

Compartilhamento de parâmetros:

- Redução do número de parâmetros.
- Processamento de sequências de tamanhos diferentes.

Recapitulação

Redes neurais recorrentes

- Problemas
  - Desvanecimento de gradientes [*vanishing gradients*]
  - Explosão de gradientes [*exploding gradients*]
  - Processamento sequencial (lento)
  - Estrangulamento da capacidade / gargalo [*bottleneck problem*]
- Arquiteturas baseadas em *long short-term memory* (LSTMs) e *gated recurrent units* (GRUs) mitigam problemas que ocorrem em RNNs clássicas.
  - Melhoram a modelagem de dependências de longo prazo (porém ainda de forma limitada).
  - Endereçam o problema do desvanecimento/explosão de gradientes sofridos pelas RNNs.
  - Porém são mais lentas para treinar e não permitem processamento paralelo.

Por essas razões, modelos baseados em redes recorrentes foram largamente suplantados por modelos baseados em ***transformers***.

Transformadores

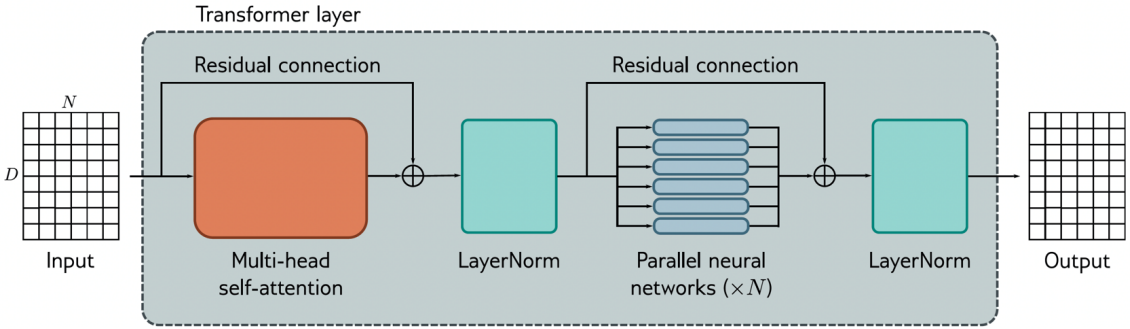
Introdução

- Representam um dos desenvolvimentos recentes mais importantes em aprendizado profundo.
- Baseiam-se no **mecanismo de atenção**:
  - Permite atribuir pesos diferentes a entradas diferentes, com coeficientes que dependem dos próprios valores de entrada.
- Atuam como **transformadores de representações**:

$$\boldsymbol{X} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\} \xrightarrow{\textit{TransformerLayer}} \tilde{\boldsymbol{X}} = \{\tilde{\boldsymbol{x}}_1, \ldots, \tilde{\boldsymbol{x}}_N\}$$

Transformadores

Introdução



S. J. D. Prince. *Understanding Deep Learning*, The MIT Press, 2023.

Transformadores

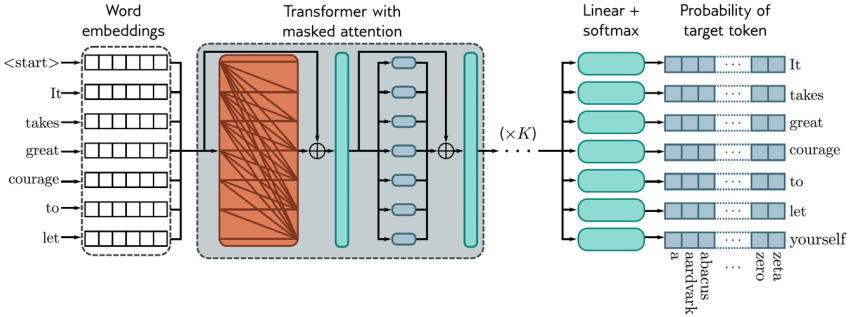
Introdução

- Atuam como **transformadores de representações**:
  - Entrada ($\boldsymbol{X}$): conjunto de vetores em um certo espaço de representação.
  - Saída ($\tilde{\boldsymbol{X}}$): novo conjunto de vetores, mesma dimensionalidade, em um espaço de representação mais rico para resolver tarefas subsequentes.

Transformadores

Introdução

- Origem na área de processamento de linguagem natural (PLN):
  - Linguagem natural = português, inglês, francês, espanhol, etc.
  - Suplantaram o estado da arte anterior baseado em RNNs.



S. J. D. Prince. *Understanding Deep Learning*, The MIT Press, 2023.

Transformadores

Introdução

- Expansão para outros domínios:
  - *Vision transformers* na área de visão computacional;
  - Transformadores multimodais combinam dados de diferentes naturezas, por exemplo:
    - texto
    - imagens
    - áudio
    - vídeo

Transformadores

Introdução

Hipótese de escala

Aumentar o número de parâmetros treináveis de um modelo e o tamanho do conjunto de dados leva a melhorias significativas de desempenho, mesmo sem mudanças na arquitetura.

Transformadores

Introdução

- Possuem arquitetura particularmente adequada à paralelização (e portanto ao uso de hardware paralelo como *graphical processing units* - GPUs) permitindo o treinamento de grandes modelos.
- Podem ser treinados de forma **auto-supervisionada** utilizando quantidades massivas de dados não rotulados.
  - **Aprendizado auto-supervisionado**: busca aprender um mapeamento entrada $\mapsto$ saída onde os rótulos são gerados automaticamente a partir dos dados de treinamento (isto é, sem rotulação manual).
  - Grandes volumes de texto estão disponíveis em múltiplas fontes (ex.: internet); esse regime permite escalar para conjuntos de treinamento massivos e redes neurais de grande porte.
- ***Foundation model***: modelo de grande escala que pode ser posteriormente adaptado para resolver múltiplas tarefas diferentes.

Transformadores

Introdução

Objetivo

Gerar representações mais ricas, modelar melhor dependências de longo alcance e usufruir de paralelismo no treinamento.

Transformadores

Mecanismo de Atenção

- A arquitetura de transformadores é construída em torno do **mecanismo de atenção**.
- Evolução histórica:
  1. **"Mecanismo auxiliar"**
     introduzido como um aprimoramento de RNNs para a tarefa de tradução automática.
  2. **Remoção da recorrência**
     posteriormente, mostrou-se que era possível eliminar a estrutura recorrente e obter desempenho significativamente melhor usando apenas o mecanismo de atenção.
  3. **Domínio dos transformadores**
     atualmente, transformadores suplantaram RNNs em praticamente todas as aplicações.



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* † illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Vaswani et al. (2017), Attention is All You Need, NeurIPS 30.

Transformadores

Mecanismo de Atenção

Exemplo em linguagem natural. Considere as sentenças:

1. *I swam across the river to get to the other bank.*
   [Atravessei o rio a nado para chegar à outra margem.]
   - Significado de bank: margem de um rio
2. *I walked across the road to get cash from the bank.*
   [Atravessei a rua para sacar dinheiro no banco.]
   - Significado de bank: instituição financeira.

Como inferir o significado correto de *bank*?

Transformadores

Mecanismo de Atenção

Exemplo em linguagem natural. Considere as sentenças:

1. *I swam across the river to get to the other bank.*
   [Atravessei o rio a nado para chegar à outra margem.]
   - Significado de bank: margem de um rio
2. *I walked across the road to get cash from the bank.*
   [Atravessei a rua para sacar dinheiro no banco.]
   - Significado de bank: instituição financeira.

Como inferir o significado correto de *bank*?

O significado correto de *bank* só pode ser inferido a partir do **contexto** fornecido pelo restante da sequência.

Transformadores

Mecanismo de Atenção

Nem todas as palavras da sequência têm a mesma importância para resolver a ambiguidade:

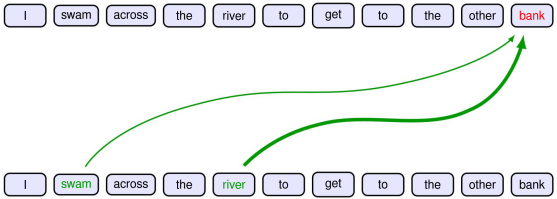
1. *I swam across the river to get to the other bank.*
   - Palavras como "*swam*" [nadei] e "*river*" [rio] são fortes indicadores de que *bank* se refere à margem de um rio.
2. *I walked across the road to get cash from the bank.*
   - A palavra "*cash*" [dinheiro] indica fortemente que *bank* se refere a uma instituição financeira.

Transformadores

Mecanismo de Atenção

Ideia central do mecanismo de atenção

Ao processar um elemento (ex.: a palavra *bank*), a rede neural deve **"prestar mais atenção"** a certas palavras da sequência, isto é, atribuir maior peso a alguns elementos do que a outros. As posições que devem receber mais atenção dependem da **sequência específica**.



Transformadores

Mecanismo de Atenção

C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Transformadores

Mecanismo de Atenção

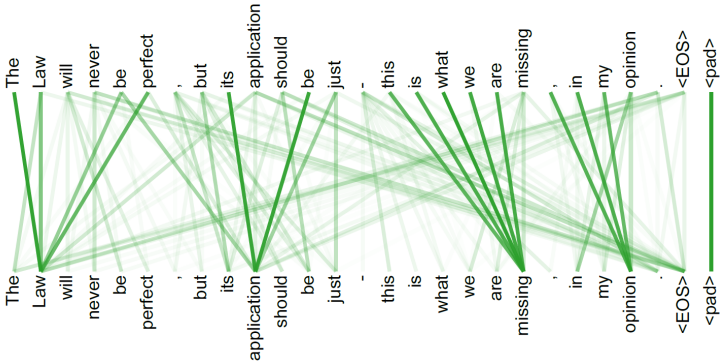
Redes neurais "padrão" vs. Mecanismo de atenção:

- Redes neurais "padrão": após o treinamento, os pesos que multiplicam cada entrada são **fixos**.
- Mecanismo de atenção: os *pesos de atenção* **dependem da entrada**.

Transformadores

Mecanismo de Atenção

- Em um transformador treinado em linguagem natural, esses **pesos de atenção** indicam quanto cada posição da sequência "olha" para as demais.



Vaswani et al. (2017), Attention is All You Need, NeurIPS 30.

Transformadores

Mecanismo de Atenção

Em PLN, utilizamos ***embeddings* de palavras** [*word embeddings*], os quais capturam propriedades semânticas das mesmas:

- Palavra $\xrightarrow{\text{mapeamento}}$ vetor em um espaço de *embedding*.
- Palavras com significados semelhantes tendem a ficar em regiões próximas nesse espaço.
- A mesma palavra é sempre mapeada para o mesmo vetor de embedding (independentemente do contexto).

Transformadores

Mecanismo de Atenção

Transformadores como geradores de **embeddings contextuais**:

- Um transformador pode ser visto como uma forma de gerar representações vetoriais mais ricas, em que o vetor transformado para um dado elemento depende dos vetores associados aos outros elementos da sequência.
- Ex.: a palavra *bank* pode ser transformada em vetores diferentes dentro de um novo espaço de representação, a depender do contexto em que aparece:
  1. *I swam across the river to get to the other bank.*
     A representação transformada de *bank* pode ficar "próxima" de *water* [água].
  2. *I walked across the road to get cash from the bank.*
     A representação transformada de *bank* pode ficar "próxima" de *money* [dinheiro].

Transformadores

Processamento

- Entrada de um transformador: conjunto de vetores

$$\{\boldsymbol{x}_n\}_{n=1}^{N}, \quad \boldsymbol{x}_n \in \mathbb{R}^D$$

  - $\boldsymbol{x}_n = [x_{n,1}, \ldots, x_{n,D}]^\top$: n-ésimo ***token***.
  - Ex.: 1 palavra em uma sentença, 1 região (*patch*) em uma imagem, ...
  - $x_{n,d}$: d-ésima **característica** [*feature*] de $\boldsymbol{x}_n$.

- Empilhamos os N tokens em uma matriz

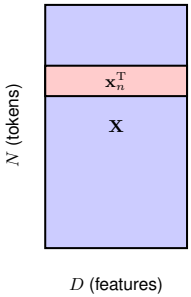
$$\boldsymbol{X} \in \mathbb{R}^{N \times D}$$

  - $\boldsymbol{X} = [x_{n,d}]_{N \times D}$: conjunto de *tokens* de entrada.

- Conjunto de dados:

$$\mathcal{D} = \{\boldsymbol{X}_m\}_{m=1}^{M}$$

  - Ex.: vários trechos independentes de texto.



C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Transformadores

Processamento

- Camada transformadora [*transformer layer*]: recebe $\boldsymbol{X}$ e produz uma matriz transformada $\tilde{\boldsymbol{X}}$ com a mesma dimensionalidade:

$$\tilde{\boldsymbol{X}} = \texttt{TransformerLayer}[\boldsymbol{X}], \quad \boldsymbol{X}, \tilde{\boldsymbol{X}} \in \mathbb{R}^{N \times D}.$$

  - Múltiplas camadas em sequência formam redes neurais profundas que aprendem representações internas mais ricas.
  - Cada camada possui seus próprios parâmetros (pesos e vieses), ajustados por gradiente descendente usando uma função de perda apropriada.

Transformadores

Processamento

Uma camada *transformer* é composta por duas etapas:

1. **Etapa 1 — mecanismo de atenção**:
   - mistura as mesmas características (mesma coluna) de *tokens* diferentes;
   - isto é, combina informação ao longo das linhas de $\boldsymbol{X}$ (entre *tokens*).
2. **Etapa 2 — transformação por *token***:
   - atua independentemente em cada linha (cada token) de $\boldsymbol{X}$;
   - isto é, transforma as características dentro de cada *token*.

Transformadores

Coeficientes de Atenção

Queremos construir o mapeamento:

$$\{\boldsymbol{x}_n\}_{n=1}^N \mapsto \{\boldsymbol{y}_n\}_{n=1}^N$$

- $\{\boldsymbol{x}_n\}_{n=1}^N$: N *tokens* de entrada em um certo espaço de *embedding*
- $\{\boldsymbol{y}_n\}_{n=1}^N$: N *tokens* de saída em um novo espaço de embedding que capture uma estrutura semântica mais rica.
  - Cada vetor de saída $\boldsymbol{y}_n$ deve depender de todos os tokens de entrada $\{\boldsymbol{x}_n\}_{n=1}^N$ e não apenas do *token* correspondente $\boldsymbol{x}_n$.
  - **Mecanismo de atenção**: a influência de cada token de entrada $\boldsymbol{x}_m$ sobre a saída $\boldsymbol{y}_n$ deve ser **maior** para aqueles tokens mais importantes para determinar a nova representação de $\boldsymbol{y}_n$.

Como implementar o mecanismo de atenção?

Transformadores

Coeficientes de Atenção

Uma forma simples de formalizar isso é definir cada vetor de saída como uma **combinação linear** dos vetores de entrada:

$$\boldsymbol{y}_n = \sum_{m=1}^{N} a_{nm}\, \boldsymbol{x}_m,$$

- a_{nm}: **peso de atenção** associado à saída n e à entrada m.
- $\uparrow a_{nm}$: *tokens* relevantes para $\boldsymbol{y}_n$.
- $\downarrow a_{nm}$: *tokens* pouco relevantes para $\boldsymbol{y}_n$.

Transformadores

Coeficientes de Atenção

Restrições sobre os coeficientes a_{nm}:

- Evitar que um coeficiente se torne grande e positivo e outro grande e negativo apenas para se compensarem:

$$a_{nm} \geq 0$$

- "Toda escolha é uma renúncia": "prestar mais atenção" a um *token* deve implicar "prestar menos atenção" aos demais:

$$\sum_{m=1}^{N} a_{nm} = 1$$

Transformadores

Coeficientes de Atenção

Para cada vetor de saída $\boldsymbol{y}_n$:

- Existe um conjunto próprio de coeficientes $\{a_{nm}\}_{m=1}^{N}$.
- Cada coeficiente satisfaz $0 \leq a_{nm} \leq 1$.
- O conjunto $\{a_{nm}\}_{m=1}^{N}$ define uma **partição da unidade**.
- Os coeficientes a_{nm} **dependem da entrada**.
- As restrições sobre os coeficientes de atenção são aplicadas separadamente para cada $n \in \{1, \ldots, N\}$.

De forma geral, $\boldsymbol{y}_m$ é uma **mistura** dos vetores de $\{\boldsymbol{x}_n\}_{n=1}^{N}$, com alguns recebendo mais peso que outros.

- Se $a_{mm} = 1$ e $a_{nm} = 0$ para $n \neq m$, então $\boldsymbol{y}_m = \boldsymbol{x}_m$.

Transformadores

Autoatenção

Como determinar os coeficientes de atenção a_{nm}?

Transformadores

Autoatenção

Analogia com a área de **recuperação de informação** [*information retrieval*]: considere a escolha de um filme em um serviço de *streaming*. Assuma que cada filme é descrito por um vetor de atributos:

- gênero
- atores
- duração
- etc.

Transformadores

Autoatenção

- Na analogia com sistemas de busca:
  - **Chave** [*key*]: vetor que codifica os atributos de cada filme.
  - **Valor** [*value*]: o próprio filme correspondente a uma dada chave.
  - **Consulta** [*query*]: vetor fornecido pelo usuário, representando suas preferências (gênero, atores, etc.).
- O sistema compara a consulta com todas as chaves:
  - escolhe a chave mais próxima da consulta;
  - retorna o valor associado (o filme).
- Podemos interpretar isso como o usuário "prestando atenção" ao filme cuja chave mais se aproxima de sua consulta.

Transformadores

Autoatenção

- No exemplo de filmes, o sistema retorna um único filme (um único valor) → **atenção rígida** [*hard attention*].
- Transformadores → **atenção suave** [*soft attention*]:
  - variáveis contínuas medem o grau de aderência entre consultas [*queries*] e chaves [*keys*];
  - essas variáveis são então usadas para ponderar a influência dos valores [*values*] nas saídas.
  - Isso garante que a função implementada pelo transformador seja **diferenciável** e possa ser treinada pelo gradiente descendente.

Transformadores

Autoatenção

- Adaptação da analogia para o transformador:
  - vetor de entrada $\boldsymbol{x}_n$:
    - **vetor de valor** que será utilizado para formar os *tokens* de saída.
    - **vetor de chave** do n-ésimo *token* de entrada.
    - **vetor de consulta** para a saída $\boldsymbol{y}_n$.
- Intuição:
  - Para saber o quanto o token $\boldsymbol{x}_n$ deve "**prestar atenção**" a um token $\boldsymbol{x}_m$, precisamos medir a **similaridade** entre esses vetores.

Como medir a similaridade?

Transformadores

Autoatenção

- Uma medida simples de similaridade entre dois vetores é o **produto escalar**:

$$\boldsymbol{x}_n^\top \boldsymbol{x}_m.$$

- Para impor as restrições sobre os coeficientes de atenção ($a_{nm} \geq 0$ e $\sum_m a_{nm} = 1$), utilizamos a função `softmax`:

$$a_{nm} = \texttt{softmax}(\boldsymbol{x}_n^\top \boldsymbol{x}_m) = \frac{\exp(\boldsymbol{x}_n^\top \boldsymbol{x}_m)}{\sum_{m'=1}^{N} \exp(\boldsymbol{x}_n^\top \boldsymbol{x}_{m'})}.$$

  - A `softmax()` é usada para **normalizar** apropriadamente os pesos de atenção e não tem interpretação probabilística aqui.

Transformadores

Autoatenção

- Cada vetor de saída $\boldsymbol{y}_n$ (transformação de $\boldsymbol{x}_n$) é dado por uma combinação linear

$$\boldsymbol{y}_n = \sum_{m=1}^{N} a_{nm} \ \boldsymbol{x}_m,$$

onde $a_{nm} = \texttt{softmax}(\boldsymbol{x}_n{}^\top \boldsymbol{x}_m)$.

  - $\boldsymbol{x}_n$: **consulta** para a saída n
  - $\boldsymbol{x}_m$: **chave** para a entrada m
  - $\boldsymbol{x}_m$: **valor** utilizado para gerar a saída

- Caso particular:
  - Se todos os vetores de entrada forem ortogonais entre si, então cada saída coincide com a entrada correspondente:

$$\boldsymbol{y}_m = \boldsymbol{x}_m, \quad \forall m \in \{1, \ldots, N\}.$$

Transformadores

Autoatenção

Forma matricial:

$$\boldsymbol{Y} = \texttt{softmax}(\boldsymbol{X}\boldsymbol{X}^\top)\boldsymbol{X}$$

- $\boldsymbol{X} \in \mathbb{R}^{N \times D}$: matriz de entrada
- $\boldsymbol{Y} \in \mathbb{R}^{N \times D}$: matriz de saída (linhas $\boldsymbol{y}_n^\top$).
- $\texttt{softmax}(\boldsymbol{L})$: normaliza cada linha da matriz $\boldsymbol{L}$ independentemente.

Transformadores

Autoatenção

Terminologia:

- **Autoatenção** [*self-attention*]: a mesma sequência de vetores é usada para definir
  - **consultas**;
  - **chaves**;
  - **valores**.
- **Mecanismo de autoatenção por produto escalar** [*dot-product self-attention*]: a similaridade entre consulta e chave é dada por um produto escalar.

Transformadores

Parâmetros da Rede

- Até aqui, a transformação

$$\{\boldsymbol{x}_n\}_{n=1}^N \mapsto \{\boldsymbol{y}_n\}_{n=1}^N$$

é **fixa**, sem parâmetros treináveis:

  - Incapaz de **aprender com dados**;
  - Cada característica de $\boldsymbol{x}_n$ contribui de forma idêntica para os coeficientes de atenção.

- Queremos:
  1. Introduzir **parâmetros treináveis**;
  2. Permitir que a rede **dê mais peso** a certas características ao definir a similaridade entre tokens.

Como podemos alcançar nosso objetivo?

Transformadores

Parâmetros da Rede

- Podemos aplicar uma transformação linear:

$$\tilde{\boldsymbol{X}} = \boldsymbol{X}\boldsymbol{U},$$

onde:

  - $\boldsymbol{X} \in \mathbb{R}^{N \times D}$: matriz de entrada;
  - $\boldsymbol{U} \in \mathbb{R}^{D \times D}$: matriz de parâmetros treináveis.

- A autoatenção passa a ter a forma:

$$\boldsymbol{Y} = \texttt{softmax}(\boldsymbol{X}\boldsymbol{U}\boldsymbol{U}^\top\boldsymbol{X}^\top)\boldsymbol{X}\boldsymbol{U}$$

Mas quais são os problemas?

Transformadores

Parâmetros da Rede

Limitações da formulação $\tilde{\boldsymbol{X}} = \boldsymbol{X}\boldsymbol{U}$:

1. A matriz $\boldsymbol{X}\,\boldsymbol{U}\,\boldsymbol{U}^\top\boldsymbol{X}^\top$ é **simétrica**. Queremos que o mecanismo de atenção modele relações **assimétricas**. Por exemplo, considere a associação entre *formão* e *ferramenta*:
   - Todo formão é uma ferramenta: formão deve estar fortemente associado a ferramenta.
   - Nem toda ferramenta é um formão: ferramenta deve estar fracamente associada a formão.
2. Restrições quanto à flexibilidade do modelo
   - A mesma matriz $\boldsymbol{U}$ é usada para definir os **valores** e para os **coeficientes de atenção** (**consultas** e **chaves**).

$$\boldsymbol{Y} = \underbrace{\texttt{softmax}(\underbrace{\boldsymbol{X}\boldsymbol{U}}_{\text{consultas}}\ \underbrace{\boldsymbol{U}^\top\boldsymbol{X}^\top}_{\text{chaves}})}_{\text{coeficientes de atenção}}\ \underbrace{\boldsymbol{X}\boldsymbol{U}}_{\text{valores}}$$

Transformadores

Parâmetros da Rede

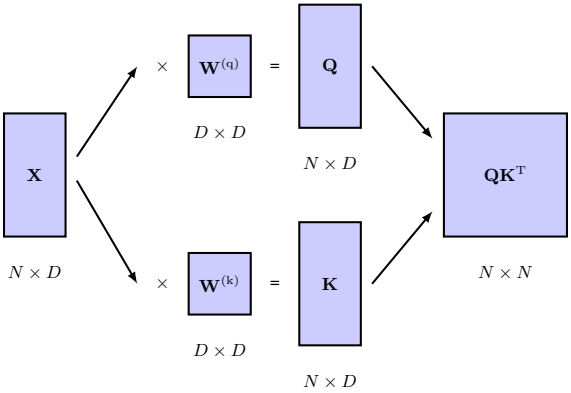
- **Solução**: introduzir três transformações lineares distintas, com parâmetros independentes:

$$\boldsymbol{Q} = \boldsymbol{X}\,\boldsymbol{W}^{(q)}, \quad \boldsymbol{K} = \boldsymbol{X}\,\boldsymbol{W}^{(k)}, \quad \boldsymbol{V} = \boldsymbol{X}\,\boldsymbol{W}^{(v)}.$$

  - $\boldsymbol{Q}$: matriz de **consultas**;
  - $\boldsymbol{K}$: matriz de **chaves**;
  - $\boldsymbol{V}$: matriz de **valores**.
- Parâmetros treináveis:
  - $\boldsymbol{W}^{(k)} \in \mathbb{R}^{D \times D_k}$;
  - $\boldsymbol{W}^{(q)} \in \mathbb{R}^{D \times D_k}$;
  - $\boldsymbol{W}^{(v)} \in \mathbb{R}^{D \times D_v}$.
  - Escolha típica: $D_k = D_v = D$.

Transformadores

Parâmetros da Rede



C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Transformadores

Parâmetros da Rede

Generalização da forma matricial da autoatenção:

$$\boldsymbol{Y} = \texttt{softmax}(\boldsymbol{Q}\boldsymbol{K}^\top)\boldsymbol{V}$$

- $\boldsymbol{Q}\,\boldsymbol{K}^\top \in \mathbb{R}^{N\times N}$: matriz que determina os coeficientes de atenção;
- $\boldsymbol{Y} \in \mathbb{R}^{N\times D_v}$: matriz de saídas (uma linha por *token*).

Na prática, as transformações lineares podem incluir termos de **viés**:

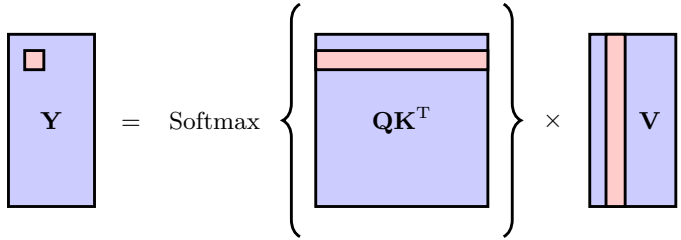
- Os vieses podem ser absorvidos nas matrizes de pesos ao se adicionar uma coluna de 1's em $\boldsymbol{X}$ e uma linha extra em $\boldsymbol{W}^{(q)}$, $\boldsymbol{W}^{(k)}$, $\boldsymbol{W}^{(v)}$.

Transformadores

Parâmetros da Rede

Coeficientes de atenção (A):

$$A = \texttt{softmax}\left(Q\,K^\top\right) \qquad a_{i,m} = \frac{\exp(q_i^\top\, k_m)}{\sum_{n=1}^{N} \exp(q_i^\top\, k_n)}$$



C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Transformadores

Parâmetros da Rede

Transformadores vs. redes neurais convencionais:

- **Redes neurais convencionais**: ativações são multiplicadas por pesos fixos após o treinamento;
  - "Ignorar" uma entrada ou unidade oculta significa ignorá-la sempre.
- **Transformadores**: ativações são multiplicadas por coeficientes de atenção que dependem dos dados.
  - se, para uma certa entrada, um coeficiente de atenção for próximo de zero, o caminho de sinal correspondente é efetivamente ignorado para aquela entrada específica.

Transformadores

Autoatenção Escalonada

Problema:

- Os gradientes da função `softmax()` ficam **exponencialmente pequenos** quando seus argumentos têm magnitude muito alta (fenômeno análogo ao observado com $\tanh$ ou sigmoide logística).

Solução:

- **Autoatenção por produto escalar escalonada** [*scaled dot-product self-attention*]: o produto $\boldsymbol{Q}\,\boldsymbol{K}^\top$ é normalizado por $\sqrt{D_k}$ antes de aplicar a `softmax()`:

$$\boldsymbol{Y} = \texttt{attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) \triangleq \texttt{softmax}\left(\frac{\boldsymbol{Q}\,\boldsymbol{K}^\top}{\sqrt{D_k}}\right)\boldsymbol{V}$$

Transformadores

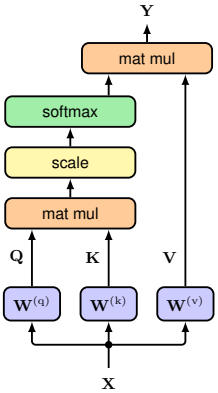
Autoatenção Escalonada

Camada de autoatenção (1 cabeça)

$$\boldsymbol{Y} = \texttt{attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \texttt{softmax}\left(\frac{\boldsymbol{Q}\,\boldsymbol{K}^\top}{\sqrt{D_k}}\right)\boldsymbol{V}$$

onde:

- Consultas: $\boldsymbol{Q} = \boldsymbol{X}\,\boldsymbol{W}^{(q)}, \quad \boldsymbol{Q} \in \mathbb{R}^{N \times D_k}$
- Chaves: $\boldsymbol{K} = \boldsymbol{X}\,\boldsymbol{W}^{(k)}, \quad \boldsymbol{K} \in \mathbb{R}^{N \times D_k}$
- Valores: $\boldsymbol{V} = \boldsymbol{X}\,\boldsymbol{W}^{(v)}, \quad \boldsymbol{V} \in \mathbb{R}^{N \times D_v}$
- Conjunto de *tokens*: $\boldsymbol{X} \in \mathbb{R}^{N \times D}$
- Parâmetros treináveis:
  - $\{\boldsymbol{W}^{(q)}, \boldsymbol{W}^{(k)}\} \in \mathbb{R}^{D \times D_k}$ e $\boldsymbol{W}^{(v)} \in \mathbb{R}^{D \times D_v}$
- Saída: $\boldsymbol{Y} \in \mathbb{R}^{N \times D_v}$



C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Transformadores

Atenção multi-cabeças

- Múltiplos padrões de atenção podem ser simultaneamente relevantes.
  - Por exemplo, em linguagem natural:
    - Alguns padrões podem estar relacionados ao **tempo verbal**;
    - Outros podem estar associados ao **vocabulário**.
- **Problema**: Uma única cabeça de atenção pode levar a uma média desses padrões de atenção.
- **Solução**: usar múltiplas cabeças em paralelo com parâmetros independentes para consultas, chaves e valores.
  - Analogia: múltiplos filtros em cada camada de uma rede neural convolucional.

Transformadores

Atenção multi-cabeças

No mecanismo de atenção multi-cabeças cada cabeça computa:

$$\boldsymbol{H}_h = \texttt{attention}(\boldsymbol{Q}_h, \boldsymbol{K}_h, \boldsymbol{V}_h),$$

- Índice da cabeça: $h \in \{1, \ldots, H\}$
- Autoatenção escalonada: `attention()`
- As matrizes de cada cabeça são definidas por transformações lineares independentes:

$$\boldsymbol{Q}_h = \boldsymbol{X}\boldsymbol{W}_h^{(q)}, \quad \boldsymbol{K}_h = \boldsymbol{X}\boldsymbol{W}_h^{(k)}, \quad \boldsymbol{V}_h = \boldsymbol{X}\boldsymbol{W}_h^{(v)}.$$

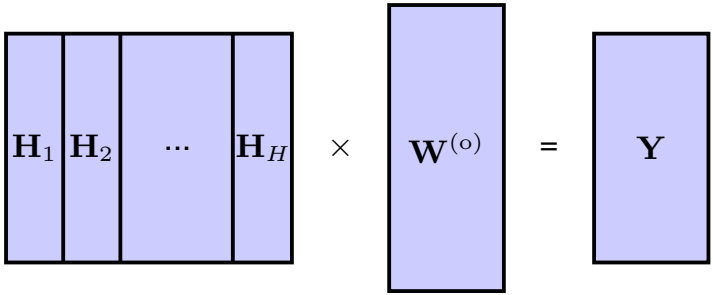
Transformadores

Atenção multi-cabeças

- Combinação das cabeças de atenção:
  1. Concatenação
  $$\texttt{concat}[\boldsymbol{H}_1, \ldots, \boldsymbol{H}_H]$$
  2. Transformação linear
  $$\boldsymbol{Y}(\boldsymbol{X}) = \texttt{concat}[\boldsymbol{H}_1, \ldots, \boldsymbol{H}_H]\boldsymbol{W}^{(o)}$$
- Dimensionalidades:
  - $\boldsymbol{H}_h \in \mathbb{R}^{N \times D_v}$ para $h \in \{1, \ldots, H\}$
  - $\texttt{concat}[\boldsymbol{H}_1, \ldots, \boldsymbol{H}_H] \in \mathbb{R}^{N \times HD_v}$
  - $\boldsymbol{W}^{(o)} \in \mathbb{R}^{HD_v \times D}$
  - $\boldsymbol{Y} \in \mathbb{R}^{N \times D}$
  - D_v é geralmente escolhido como $D_v = D/H$ de modo que a concatenação tenha dimensão $N \times D$.
- Parâmetros treináveis: $\boldsymbol{W}^{(o)}$ e $\{\boldsymbol{W}_h^{(q)}, \boldsymbol{W}_h^{(k)}, \boldsymbol{W}_h^{(v)}\}_{h=1}^{H}$.

Transformadores

Atenção multi-cabeças



C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Transformadores

Atenção multi-cabeças

Camada de autoatenção (múltiplas cabeças)

$$\boldsymbol{H}_h = \texttt{attention}(\boldsymbol{Q}_h, \boldsymbol{K}_h, \boldsymbol{V}_h), \quad h \in \{1, \ldots, H\}$$

$$\boldsymbol{H} = \texttt{Concat}[\boldsymbol{H}_1, \ldots, \boldsymbol{H}_H]$$

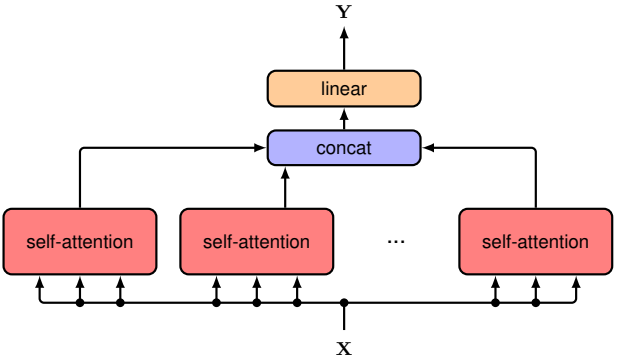
$$\boldsymbol{Y} = \boldsymbol{H}\boldsymbol{W}^{(o)}$$

- Consultas (cabeça h): $\boldsymbol{Q}_h = \boldsymbol{X}\boldsymbol{W}_h^{(q)}, \quad \boldsymbol{Q}_h \in \mathbb{R}^{N \times D}$
- Chaves (cabeça h): $\boldsymbol{K}_h = \boldsymbol{X}\boldsymbol{W}_h^{(k)}, \quad \boldsymbol{K}_h \in \mathbb{R}^{N \times D}$
- Valores (cabeça h): $\boldsymbol{V}_h = \boldsymbol{X}\boldsymbol{W}_h^{(v)}, \quad \boldsymbol{V}_h \in \mathbb{R}^{N \times D_v}$
- Conjunto de *tokens*: $\boldsymbol{X} \in \mathbb{R}^{N \times D}$
- Saída intermediária por cabeça: $\boldsymbol{H}_h \in \mathbb{R}^{N \times D_v}$
- Concatenação das cabeças: $\boldsymbol{H} \in \mathbb{R}^{N \times H D_v}$
- Saída: $\boldsymbol{Y} \in \mathbb{R}^{N \times D}$
- Parâmetros treináveis:
  - $\boldsymbol{W}_h^{(v)} \in \mathbb{R}^{D \times D_v}$ e $\{\boldsymbol{W}_h^{(q)}, \boldsymbol{W}_h^{(k)}\} \in \mathbb{R}^{D \times D}, \quad h = 1, \ldots, H$
  - $\boldsymbol{W}^{(o)} \in \mathbb{R}^{H D_v \times D}$

Transformadores

Atenção multi-cabeças

Camada de autoatenção de múltiplas cabeças



C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Camadas na arquitetura do Transformador

Estrutura de uma camada

Camada típica da arquitetura de um transformador:

$$\tilde{\boldsymbol{X}} = \texttt{TransformerLayer}(\boldsymbol{X}), \quad \boldsymbol{X}, \tilde{\boldsymbol{X}} \in \mathbb{R}^{N \times D}.$$

- Múltiplas camadas desse tipo são empilhadas;
- Em geral, as camadas têm estrutura idêntica;
- Não há compartilhamento de pesos e vieses entre camadas diferentes.

1. Atenção de múltiplas cabeças:

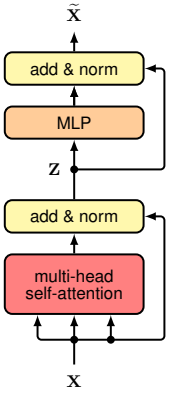
$$\boldsymbol{Y} = \texttt{MultiHeadAttention}(\boldsymbol{X})$$

2. Conexão residual e normalização por camada

$$\boldsymbol{Z} = \texttt{LayerNorm}(\boldsymbol{Y} + \boldsymbol{X}), \quad \boldsymbol{Z} \in \mathbb{R}^{N \times D}$$

3. Rede perceptron de múltiplas camadas (compartilhada)

$$\tilde{\boldsymbol{X}} = \texttt{LayerNorm}(\texttt{MLP}(\boldsymbol{Z}) + \boldsymbol{Z})$$



C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Camadas na arquitetura do Transformador

Estrutura de uma camada

- **Conexões residuais** são utilizadas para melhorar eficiência de treinamento.
  - Isso exige que $\boldsymbol{X}, \boldsymbol{Y} \in \mathbb{R}^{N \times D}$.
- **Problema**: limitação da capacidade expressiva da camada de atenção.
  - As saídas de uma camada de atenção são **combinações lineares** das entradas.
  - Apesar da dependência não linear via `softmax()`, os vetores de saída permanecem confinados ao subespaço gerado pelos vetores de entrada.
- **Solução**: pós-processar a saída com uma rede neural não linear padrão `MLP()` com D entradas e D saídas.
  - A mesma `MLP()` (compartilhada) é aplicada a cada vetor de saída (a cada linha de $\boldsymbol{Z}$) para lidar com sequências de comprimento variável.

Codificação posicional [*positional encoding*]

Ordem dos *tokens*

- Nas camadas do transformador, os parâmetros $\{\boldsymbol{W}_h^{(q)}, \boldsymbol{W}_h^{(k)}, \boldsymbol{W}_h^{(v)}\}$ são compartilhados entre os *tokens*.
  - Consequência: o transformador é **equivariante a permutações**
    - Permutar as linhas da entrada $\boldsymbol{X}$ produz a mesma permutação na saída $\tilde{\boldsymbol{X}}$.
  - Vantagens: permite processamento paralelo massivo e o aprendizado eficaz de dependências de curto e longo alcance.
  - Limitação em dados sequenciais:
    - A representação aprendida seria independente da ordem dos *tokens*.

Codificação posicional [*positional encoding*]

Ordem dos *tokens*

Objetivo:

- Injetar informação relativa à ordem dos *tokens* preservando a arquitetura de atenção.

Estratégia:

1. Associar um vetor de posição $\boldsymbol{r}_n$ a cada posição n;
2. Combinar $\boldsymbol{r}_n$ com o embedding $\boldsymbol{x}_n$ do *token* de entrada:

$$\hat{\boldsymbol{x}}_n = \boldsymbol{x}_n + \boldsymbol{r}_n$$

- $\boldsymbol{r}_n$ tem a mesma dimensionalidade de $\boldsymbol{x}_n$.

Por que somar $\boldsymbol{x}_n + \boldsymbol{r}_n$?

Codificação posicional [*positional encoding*]

Ordem dos *tokens*

Por que somar $x_n + r_n$?

- Mantém a dimensionalidade (concatenação elevaria custo);
- Em alta dimensão, as informações de identidade e posição dos *tokens* podem ser processadas quase separadamente pela rede neural pois vetores não correlacionados tendem a ser quase ortogonais;
- Conexões residuais ajudam a preservar a informação posicional entre camadas.

Como construir $\{r_n\}$?

Codificação posicional [*positional encoding*]

Ordem dos *tokens*

Como construir $\{r_n\}$?

- Inteiros: magnitude não limitada pode corromper o vetor de *embedding* e generalização ruim para sequências maiores do que as apresentadas durante o treinamento.
- Escala em $[0, 1]$: limitada, mas não única (depende do comprimento da sequência).

Critérios desejáveis para uma **codificação posicional**:

- codificação única por posição;
- valores limitados;
- generalização para sequências mais longas;
- codificação consistente de distâncias relativas entre *tokens*.

Codificação posicional [*positional encoding*]

Ordem dos *tokens*

- Codificação posicional senoidal:

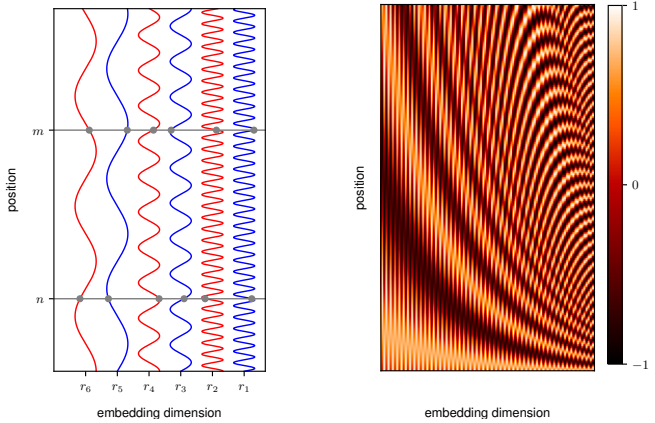
$$r_{n,i} = \begin{cases} \sin\left(\frac{n}{L^{i/D}}\right), & \text{se } i \text{ for par,} \\ \cos\left(\frac{n}{L^{(i-1)/D}}\right), & \text{se } i \text{ for ímpar.} \end{cases}$$

  - n é a posição do *token* na sequência.
  - D é a dimensionalidade do *embedding*.
  - L é um hiperparâmetro.

- Propriedades:
  - $r_{n,i} \in [-1, 1]$;
  - funções com comprimentos de onda crescentes;
  - uso conjunto de seno e cosseno.
  - o modelo pode aprender a prestar atenção a posições relativas: a codificação da posição $n + k$ pode ser escrita como uma combinação linear da codificação da posição n com coeficientes que dependem apenas de k.

Codificação posicional [*positional encoding*]

Ordem dos *tokens*



C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Codificação posicional [*positional encoding*]

Ordem dos *tokens*

- Codificações posicionais aprendidas.
  - Utilizam parâmetros treináveis;
  - Removem a equivariância a permutações (parâmetros não são compartilhados entre as posições);
  - Não generalizam para posições não vistas no treinamento;
  - Adequadas quando o comprimento de entrada é relativamente constante.

Transformadores em PLN

Categorias de modelos de linguagem

- A aplicação de transformadores na área de processamento de linguagem natural (PLN) levou ao desenvolvimento de modelos de grande escala com desempenho excepcional: grandes modelos de linguagem [*large language models*, LLMs].

Transformadores em PLN

Categorias de modelos de linguagem

- De acordo com as suas entradas e saídas, modelos de linguagem com transformadores podem ser categorizados em:
  1. *Encoder-only*: aprendizado de representação
     - Ex.: análise de sentimento.
  2. *Decoder-only*: tarefas generativas
     - Ex.: geração de texto.
  3. *Encoder-Decoder*: tarefas do tipo sequência a sequência
     - Ex.: tradução automática.

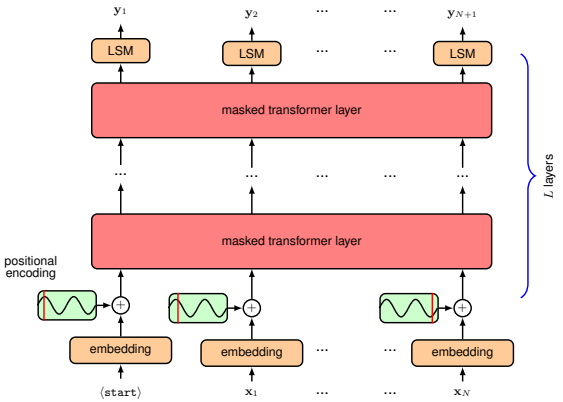
Modelos *decoder-only*

Modelagem auto-regressiva

- Modelos do tipo *decoder-only* atuam como modelos generativos.
- Exemplo canônico: Transformador pré-treinado generativo [*Generative Pretrained Transformer*, GPT].
  - Objetivo: aprender distribuições condicionais $p(x_n \mid x_{<n})$
    - Modelagem auto-regressiva via rede neural baseada em camadas transformadoras.

Arquitetura do modelo GPT

Camada de saída linear-softmax



Camada linear-softmax (LSM) compartilhada por posição:

$$\boldsymbol{Y} = \texttt{softmax}(\tilde{\boldsymbol{X}}\boldsymbol{W}^{(p)})$$

- $\boldsymbol{Y}$: distribuição sobre o vocabulário
  - linha n ($\boldsymbol{y}_n^\top$): distribuição para o *token* $n+1$
- $\boldsymbol{X} \in \mathbb{R}^{N \times D}$
- $\tilde{\boldsymbol{X}} \in \mathbb{R}^{N \times D}$
- $\boldsymbol{W}^{(p)} \in \mathbb{R}^{D \times K}$
- K: dimensionalidade do dicionário de *tokens*

C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

Treinamento do modelo GPT

Treinamento auto-supervisionado

- Treinamento **auto-supervisionado** (conjunto de dados não rotulado)
  - entrada: sequência de n *tokens*
  - saída: distribuição condicional para o *token* $n + 1$
  - alvo: *token* $n + 1$
  - função de perda: entropia cruzada por posição somada sobre o conjunto de treinamento (mini-lotes)
  - ajuste de parâmetros: algoritmo de retropropagação

Atenção causal e processamento paralelo

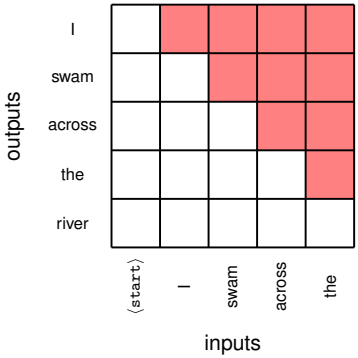
Máscaras de atenção

- Para processar a sequência inteira de uma só vez cada *token* atua simultaneamente como:
  - entrada para predições de *tokens* subsequentes;
  - alvo para a sequência de *tokens* anteriores.
- Estratégia para evitar que o modelo "trapaceie" prestando atenção a *tokens* futuros na sequência:
  1. **Deslocamento da entrada**
     - a sequência é deslocada à direita (a saída do modelo para o *token* x_n é y_{n+1} e o alvo x_{n+1});
     - adiciona-se o *token* $\langle start \rangle$ para indicar o início da sequência.
  2. **Mecanismo de atenção causal** (ou mascarada)
     - um *token* só pode prestar atenção a *tokens* anteriores.
     - Zeram-se os pesos de atenção que levariam em conta *tokens* posteriores ao *token* atual. Na prática isso significa atribuir $-\infty$ às posições proibidas, de modo que a `softmax()` retorne zero para esses valores.

Atenção causal e processamento paralelo

Máscaras de atenção

Atenção causal: pesos de atenção em vermelho são iguais a zero.



C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*, Springer, 2024.

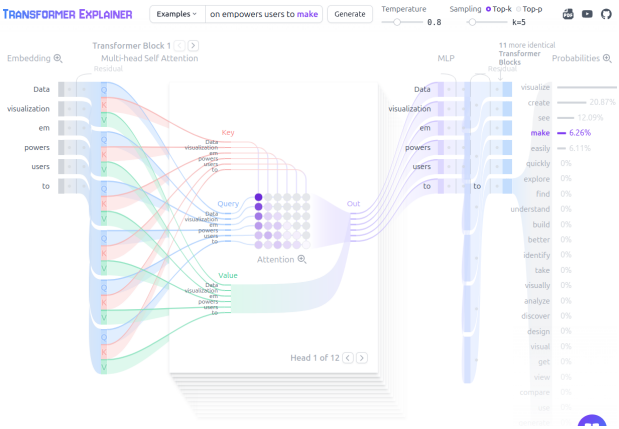
Atenção causal e processamento paralelo

Máscaras de atenção

- Para explorar o paralelismo de GPUs: queremos empilhar várias sequências no mesmo lote.
- **Problema**: sequências de texto têm comprimento variável.
- **Solução**: inserir *token(s)* de preenchimento $\langle pad \rangle$ e aplicar uma máscara de *padding* para impedir atenção às posições preenchidas.

Transformers Explainer (link)

Demonstração interativa



https://poloclub.github.io/transformer-explainer/

Geração

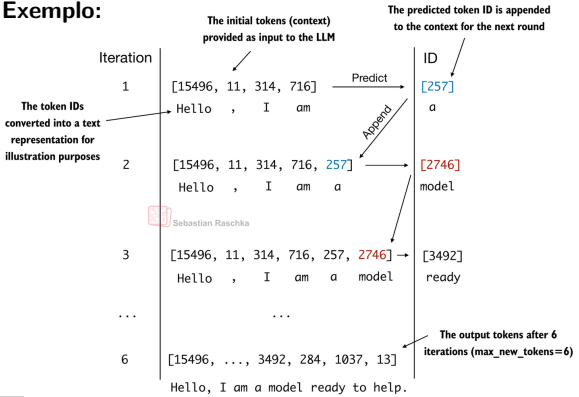
Escolha do próximo *token*

A saída do modelo é uma distribuição sobre o próximo *token*, isto é,
$p(x_n \mid \boldsymbol{x}_{<n}) = p(\text{próximo } \textit{token} \mid \text{sequência de } \textit{tokens} \text{ de entrada})$.

Geração:

1. O modelo recebe uma sequência de $n-1$ *tokens*;
2. O modelo prediz uma distribuição para o *token* n;
3. O *token* n é escolhido e anexado à sequência para estendê-la;
4. Repetem-se os passos 1–3 com a nova sequência estendida composta por n *tokens* realimentando o modelo para predizer o *token* $n+1$. O processo é repetido até o *token* que indica fim de sequência $\langle eos \rangle$ [*end-of-sequence*] ser escolhido ou limite de comprimento ser atingido.

Exemplo:



S. Raschka, *Build a Large Language Model (From Scratch)*. Manning Publications, 2024. https://sebastianraschka.com/images/LLMs-from-scratch-images/ch05_compressed/15.webp.

Geração

Escolha do próximo *token*

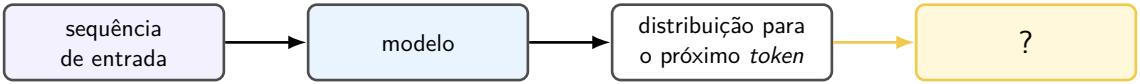
Observação

- Devido à atenção causal, a representação de um *token* depende apenas de *tokens* anteriores;
- isso permite reaproveitar parte do cálculo ao gerar *tokens* seguintes.

Geração

Escolha do próximo *token*

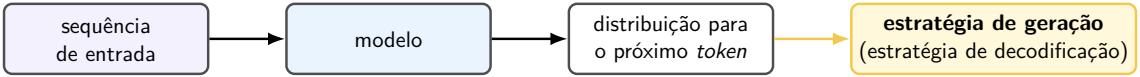
Como escolher o próximo *token* para estender a sequência?



Geração

Escolha do próximo *token*

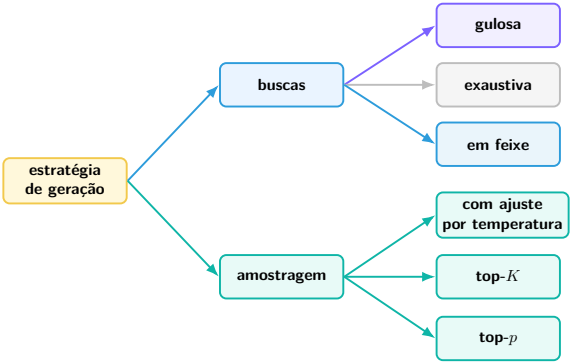
Como escolher o próximo *token* para estender a sequência?



Geração

Escolha do próximo *token*

Como escolher o próximo *token* para estender a sequência?



Busca gulosa [*greedy search*]

Estratégias de geração

Estratégia: simples

- A cada passo n, seleciona-se o *token* com maior probabilidade no vocabulário:

$$x_n = \underset{x \in \text{vocabulário}}{\arg\max}\ p(x \mid \boldsymbol{x}_{<n})$$

- Quando o modelo prediz o *token* especial $\langle eos \rangle$ ou o comprimento máximo da sequência de saída é atingido, então a sequência está completa.

Custo computacional $\mathcal{O}(KN)$: rápida

- Linear no comprimento da sequência.
  - N: número de passos na sequência;
  - K: tamanho do vocabulário.

Geração determinística: repetível

- Mesma sequência de entrada $\Rightarrow$ mesma sequência de saída.

Busca gulosa [*greedy search*]

Estratégias de geração

Observação:

decisão local

- Buscar uma sequência de *tokens* mais prováveis selecionados de maneira gulosa **não é** o mesmo que buscar a sequência mais provável!
- A sequência mais provável é aquela que maximiza a distribuição conjunta sobre todos os *tokens*:

$$p(x_1, \ldots, x_N) = \prod_{n=1}^{N} p(x_n \mid \boldsymbol{x}_{<n})$$

- Não há garantia de que a busca gulosa retorne a sequência mais provável.

Busca gulosa [*greedy search*]

Estratégias de geração

Exemplo 1:



https://langcopilot.com/posts/2025-07-02-decoding-strategies-for-large-language-models.

Busca gulosa [*greedy search*]

Estratégias de geração

Exemplo 2:

- Sequência de entrada: `todo esforço te leva`

token	perto	todo	esforço	**adiante**	pouco	leva	pizza	longe	você
logit	4.51	0.89	-1.90	**6.75**	1.63	-1.62	-1.89	6.28	1.79

- Continuação gerada: `todo esforço te leva` `adiante`
  - A `softmax` preserva a ordem dos *logits*; portanto, o maior *logit* corresponde ao *token* de maior probabilidade.
  - Nesse caso, o *token* selecionado é `adiante`.

Busca exaustiva [*exhaustive search*]

Estratégias de geração

Objetivo:
Encontrar a sequência mais provável, isto é, a sequência que maximiza

$$p(x_1, \ldots, x_N) = \prod_{n=1}^{N} p(x_n \mid \boldsymbol{x}_{<n})$$

Estratégia:

1. Enumerar todas as sequências possíveis;
2. Computar suas probabilidades;
3. Escolher a sequência mais provável.

Qual o problema da busca exaustiva?

Busca exaustiva [*exhaustive search*]

Estratégias de geração

Problema:

- Custo computacional proibitivo $\mathcal{O}(K^N)$:
  - Exponencial no comprimento da sequência N
    - Além disso, a base representada pelo tamanho do vocabulário K é enorme.
  - A busca exaustiva é impraticável.

Busca exaustiva [*exhaustive search*]

Estratégias de geração

Busca exaustiva versus busca gulosa:

Com $N = 10$ e $K = 10\,000$ (valores pequenos quando comparados com aplicações reais):

Busca exaustiva

Seria necessário avaliar $K^N = 10\,000^{10} = 10^{40}$ sequências.

Busca gulosa

É necessário avaliar somente $K \times N = 10\,000 \times 10 = 10^5$ sequências.

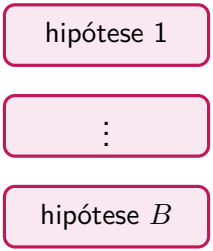
A busca exaustiva é ótima, mas inviável na prática. A busca gulosa é eficiente, mas não garante a sequência mais provável.

Busca em feixe [*beam search*]

Estratégias de geração

Estratégia:

1. Mantém-se um conjunto formado pelas B hipóteses mais prováveis (e suas respectivas probabilidades), em que B é o hiperparâmetro **largura do feixe** [*beam width*].



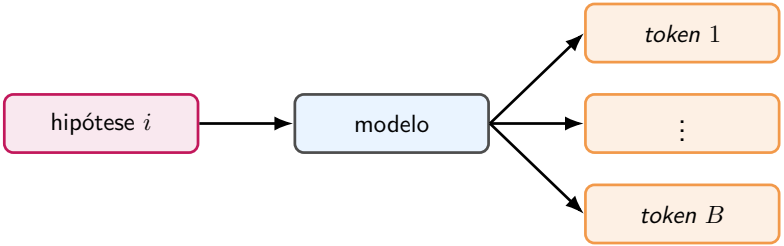
Cada hipótese representa uma sequência parcial até o passo n.

Busca em feixe [*beam search*]

Estratégias de geração

Estratégia

2. Extensão:
   - Para cada hipótese i ($i \in \{1, \ldots, B\}$):
     - O modelo calcula a distribuição para o próximo *token*;
     - Selecionam-se os B *tokens* mais prováveis como possíveis continuações (candidatas).



Busca em feixe [*beam search*]

Estratégias de geração

Estratégia

3. Extensão:
   - Isso gera B^2 hipóteses estendidas (candidatas).

hipótese candidata 1

$\vdots$

hipótese candidata B^2

Cada hipótese candidata representa uma sequência parcial até o passo $n + 1$.

Busca em feixe [*beam search*]

Estratégias de geração

Estratégia

4. Avaliação e poda: mantêm-se apenas as B sequências mais prováveis, considerando a probabilidade total da sequência estendida:

$$p(x_1, \ldots, x_{n+1}) = \prod_{i=1}^{n+1} p(x_i \mid \boldsymbol{x}_{<i})$$

5. O processo continua e no final seleciona-se a sequência mais provável dentre aquelas consideradas.

Qual o problema da abordagem acima para selecionar a sequência final?

Busca em feixe [*beam search*]

Estratégias de geração

Estratégia

4. Poda: mantêm-se apenas as B sequências mais prováveis, considerando a probabilidade total da sequência estendida:

$$p(x_1, \ldots, x_{n+1}) = \prod_{i=1}^{n+1} p(x_i \mid \boldsymbol{x}_{<i})$$

5. O processo continua e no final seleciona-se a sequência mais provável dentre aquelas consideradas.
   - Problema:
     - $p(x_i \mid \boldsymbol{x}_{<i}) \leq 1$ e $p(\boldsymbol{x}_{1:n+1}) = \prod_{i=1}^{n+1} p(x_i \mid \boldsymbol{x}_{<i}) \Rightarrow$ sequências mais longas tendem a ter probabilidade menor, portanto há viés em favor de sequências curtas.
   - Solução:
     - As probabilidades das sequências geralmente são normalizadas pelo comprimento da sequência antes da comparação.

Busca em feixe [*beam search*]

Estratégias de geração

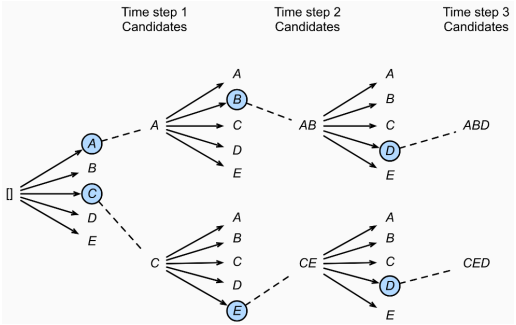
A busca em feixe:

- Explora mais sequências do que a busca gulosa, mas sem enumerar todas as sequências possíveis.
- Tem custo computacional $\mathcal{O}(BKN)$:
  - Linear no comprimento da sequência N;
  - Aumenta o custo por um fator B (custo de inferência potencialmente proibitivo para grandes modelos de linguagem).
- Oferece um equilíbrio entre a eficiência da busca gulosa e a garantia de otimalidade da busca exaustiva. Esse compromisso [*trade-off*] entre qualidade da sequência gerada e custo computacional é controlado pela largura do feixe B.
  - A busca em feixe pode encontrar sequências de maior probabilidade do que aquelas obtidas pela busca gulosa.
  - Custo computacional das buscas: gulosa $\leq$ em feixe $<$ exaustiva.
  - A busca gulosa pode ser vista como um caso especial da busca em feixe com $B = 1$.

Busca em feixe [*beam search*]

Estratégias de geração

Exemplo $(B = 2, N = 3, K = 5)$



A. Zhang et al., *Dive into Deep Learning*, Cambridge University Press, 2023.

Amostragem

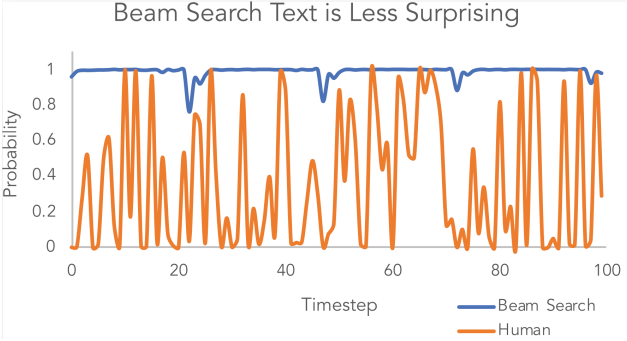
Estratégias de geração

Limitações das buscas gulosa e em feixe

- Podem reduzir a diversidade das sequências de saída.
- A geração pode ficar presa em repetições, produzindo a mesma subsequência várias vezes.
- Textos humanos podem ter menor probabilidade, isto é, serem mais "surpreendentes" para o modelo.

Amostragem

Estratégias de geração



A. Holtzman et al. "The curious case of neural text degeneration," arXiv preprint arXiv:1904.09751, 2020.

Amostragem

Estratégias de geração

Amostragem pela distribuição `softmax`

Em vez de tentar encontrar a sequência de maior probabilidade, podemos gerar os *tokens* sucessivamente por amostragem da distribuição `softmax`.

Risco

A amostragem pela distribuição `softmax` pode produzir sequências incoerentes.

- O vocabulário costuma ser muito grande.
- Há uma longa cauda de muitos *tokens* com probabilidade pequena.
- Em conjunto, esses *tokens* podem concentrar uma fração significativa da massa de probabilidade.

Amostragem

Estratégias de geração

Estratégias de amostragem que buscam um equilíbrio entre **diversidade** e **coerência**:

Amostragem top-K

Consideram-se apenas os K *tokens* com maiores probabilidades. Depois, amostra-se a partir desse conjunto restrito, usando probabilidades renormalizadas.

Amostragem top-p (ou amostragem de núcleo [*nucleus sampling*])

Calcula-se a probabilidade acumulada dos *tokens* mais prováveis até atingir um limiar. Depois, amostra-se apenas desse conjunto restrito.

Amostragem

Estratégias de geração

Amostragem com ajuste por temperatura

Outra forma de controlar a distribuição de amostragem consiste em introduzir um hiperparâmetro chamado temperatura T na `softmax`:

$$y_i = \frac{\exp(a_i/T)}{\sum_j \exp(a_j/T)}$$

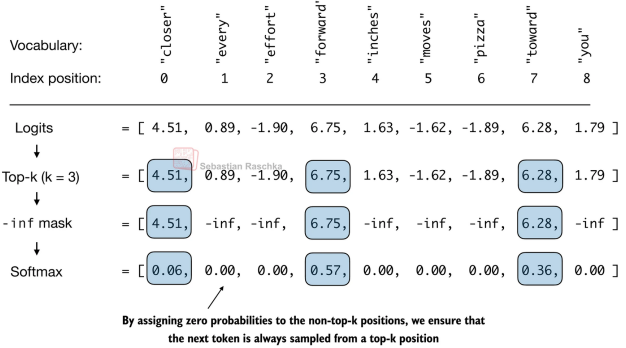
- $T \to 0$: a massa de probabilidade se concentra no *token* mais provável (aproxima a busca gulosa).
- $T = 1$: a distribuição `softmax` original.
- $T \to \infty$: a distribuição se torna uniforme.
- $0 < T < 1$: a probabilidade se concentra nos maiores valores dos *logits*.

A amostragem com ajuste de temperatura pode ser combinada por exemplo com a amostragem top-K.

Amostragem

Estratégias de geração

Exemplo: amostragem top-K



S. Raschka, *Build a Large Language Model (From Scratch)*. Manning Publications, 2024.
https://sebastianraschka.com/images/LLMs-from-scratch-images/ch05_compressed/15.webp.

Geração

Resumo

Desafio:

Durante o treinamento

O modelo recebe sequências geradas por humanos.

Durante a geração

A sequência de entrada passa a ser produzida pelo próprio modelo.

O modelo pode se afastar [*drift*] da distribuição de sequências vista durante o treinamento.

Geração

Resumo

Resumo das estratégias de geração

Busca gulosa

Descrição
Seleciona, a cada passo, o *token* de maior probabilidade.

Vantagens
Rápida e simples de implementar.

Desvantagens
Não garante a sequência mais provável; geração determinística.

Controle
—

Busca em feixe

Descrição
Mantém B hipóteses, expande cada uma delas e preserva as B mais prováveis.

Vantagens
Pode encontrar sequências de maior probabilidade do que a busca gulosa.

Desvantagens
Mais cara computacionalmente.

Controle
Largura do feixe B.

Amostragem

Descrição
Amostra o próximo *token* a partir da distribuição `softmax`.

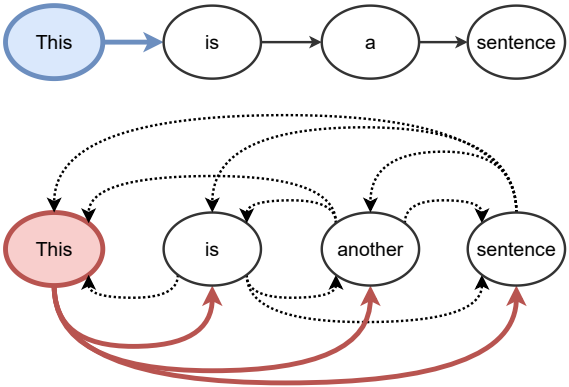
Vantagens
Pode gerar sequências mais diversas.

Desvantagens
Pode gerar sequências incoerentes devido à cauda longa de *tokens* pouco prováveis.

Controle
Temperatura, top-K e top-p.

Transformadores vs. RNNs

Processamento de sequências



RNN

Transf.

Translation?

Sentiment?

Next word?

Part-of-speech tags?

C. K. Joshi. (2025), Transformers are Graph Neural Networks, arXiv.

Referências


Simon J. D. Prince.
Understanding Deep Learning.
The MIT Press, 2023.
[Capítulo 12]
Disponível em: https://udlbook.github.io/udlbook/


Christopher M. Bishop e Hugh Bishop.
Deep Learning: Foundations and Concepts.
Springer, 2023.
[Capítulo 12]
Disponível em: https://www.bishopbook.com/

Slides construídos com base nos livros supracitados, os quais estão disponibilizados publicamente pelos seus respectivos autores.